DOCUMENTACION_MINIO_JWT

Documentación: Implementación de MinIO y JWT en UTMentor

Índice

1. MinIO: Almacenamiento de Objetos
2. JWT: Sistema de Autenticación
3. Integración y Flujo de Trabajo
4. Beneficios y Aportes al Proyecto

---

1. MinIO: Almacenamiento de Objetos

¿Qué es MinIO?

MinIO es un servidor de almacenamiento de objetos de código abierto que es compatible con la API de Amazon S3. En términos simples, es como tener tu propio servicio de almacenamiento en la nube (similar a AWS S3 o Google Cloud Storage) pero ejecutándose en tu propia infraestructura.

En el proyecto UTMentor, MinIO se utiliza para **almacenar y gestionar las fotos de perfil de los asesores**.

Configuración en Docker

MinIO está configurado como un servicio independiente en Docker Compose (`docker-compose.yml`):

```
minio:  image: quay.io/minio/minio:latest  container_name: minio_srv  restart: a
lways  environment:    MINIO_ROOT_USER: ${MINIO_ROOT_USER}    MINIO_RO
OT_PASSWORD: ${MINIO_ROOT_PASSWORD}  command: server /data --cons
ole-address ":9001"  volumes:    - minio_data:/data  ports:    - "9000:9000"    #
```

```
API S3 compatible    - "9001:9001"   # Consola web  networks:    - services_ne
t
```

Explicación de la configuración:

- **Puerto 9000**: Es la API S3 compatible, donde la aplicación se conecta programáticamente para subir y obtener archivos
- **Puerto 9001**: Es la consola web administrativa donde puedes visualizar y gestionar tus archivos manualmente
- **Volumen persistente**: `minio_data:/data` garantiza que los archivos no se pierdan cuando el contenedor se reinicia
- **Variables de entorno**: Credenciales de acceso configurables mediante archivo `.env`

Implementación en el Código

1. Configuración del Cliente MinIO (`api/config/minio.js`)

```
import * as Minio from 'minio';// Configuración del clienteconst minioClient =
new Minio.Client({
  endPoint,                                   // 'minio' en Docker, 'localhost' en des
arrollo  port: 9000,  useSSL: false,                                  // false en desarrol
lo local  accessKey: process.env.MINIO_ROOT_USER || 'minioadmin',  secretK
ey: process.env.MINIO_ROOT_PASSWORD || 'minioadmin',});export const buc
ketName = process.env.MINIO_BUCKET || 'utmentor-bucket';
```

¿Qué es un bucket?

Un bucket es como una “carpeta raíz” o “contenedor” donde se almacenan todos los objetos (archivos). En este caso, `utmentor-bucket` almacena todas las fotos de perfil.

2. Inicialización Automática (`initMinio`)

```
export async function initMinio() {
  try {
```

```
  // Verificar si el bucket existe    const exists = await minioClient.bucketExist
s(bucketName);    if (!exists) {
    // Crear el bucket automáticamente      await minioClient.makeBucket(buck
etName, 'us-east-1');      console.log(`Bucket ${bucketName} creado.`);      //
Aplicar política pública para lectura      const policy = {
      Version: "2012-10-17",       Statement: [{
        Effect: "Allow",         Principal: { AWS: ["*"] },         Action: ["s3:GetObje
ct"],         Resource: [`arn:aws:s3:::${bucketName}/*`],       }],      };      await m
inioClient.setBucketPolicy(bucketName, JSON.stringify(policy));    }
  } catch (err) {
    console.error('Error inicializando MinIO:', err);  }
}
```

¿Por qué esta función es importante?

Esta función se ejecuta automáticamente cuando el servidor arranca (ver `api/server.js`):

```
await initMinio();
```

Garantiza que:
1. El bucket exista antes de intentar subir archivos
2. El bucket tenga una política pública de lectura, permitiendo que las URLs de las fotos sean accesibles desde el navegador sin autenticación

3. Middleware de Carga de Archivos (`api/middleware/uploadMiddleware.js`)

```
import multer from 'multer';const storage = multer.memoryStorage();  // Alma
cena en memoria RAM temporalmenteconst upload = multer({
  storage: storage,  limits: {
    fileSize: 5 * 1024 * 1024,  // Máximo 5MB por imagen  },  fileFilter: (req, file,
cb) => {
    if (file.mimetype.startsWith('image/')) {
      cb(null, true);    } else {
```

```
    cb(new Error('Solo se permiten imágenes'), false);    }
  },});export default upload;
```

¿Cómo funciona Multer?

Multer es un middleware que procesa archivos enviados mediante `multipart/form-data` (formularios con archivos). En este proyecto:

1. **Almacenamiento en memoria**: Los archivos se guardan temporalmente en RAM (no en disco)
2. **Validación**: Solo acepta imágenes
3. **Límite de tamaño**: Rechaza archivos mayores a 5MB
4. **Buffer disponible**: El archivo queda en `req.file.buffer` para subirlo a MinIO

4. Controlador de Subida de Fotos (`api/controllers/usuarioController.js`)

```
export async function subirFotoPerfil(req, res) {
  const { id } = req.params;  if (!req.file) {
    return res.status(400).json({ error: "No se ha subido ningún archivo" });  }
  try {
    // 1. Crear nombre único para el archivo    const fileExtension = path.extnam
e(req.file.originalname);    const fileName = `asesores/${id}-${Date.now()}${fi
leExtension}`;    // 2. Subir a MinIO    await minioClient.putObject(
      bucketName,                // Bucket destino      fileName,                // Ruta/n
ombre del archivo      req.file.buffer,            // Contenido del archivo      req.fil
e.size,              // Tamaño      { 'Content-Type': req.file.mimetype }  // Tipo MI
ME    );    // 3. Construir URL pública    const publicHost = process.env.MINIO_
PUBLIC_HOST || 'http://localhost:9000';    const url = `${publicHost}/${bucket
Name}/${fileName}`;    // 4. Actualizar base de datos    await updateAsesorPro
file(id, { ruta_foto: url });    res.json({ message: "Foto actualizada", url });  } cat
ch (error) {
    console.error("Error al subir foto:", error);    res.status(500).json({ error: "Er
ror al procesar la imagen" });  }
}
```

Flujo completo de subida de foto:

1. El cliente envía una petición POST con el archivo
2. Multer valida y guarda el archivo en memoria
3. Se genera un nombre único usando el ID del asesor y un timestamp
4. Se sube el archivo a MinIO con `putObject`
5. Se construye la URL pública del archivo
6. Se guarda la URL en la base de datos
7. Se devuelve la URL al cliente

5. Ruta de la API (`api/routes/usuarioRoutes.js`)

```
import upload from "../middleware/uploadMiddleware.js";// Endpoint para subi
r fotorouter.post("/asesores/:id/foto", upload.single('foto'), subirFotoPerfil);
```

Uso desde el frontend:

```
const formData = new FormData();formData.append('foto', archivoSelecciona
do);fetch(`http://localhost:3000/api/usuarios/asesores/123/foto`, {
  method: 'POST',  body: formData
});
```

¿Por qué usar MinIO en lugar de guardar archivos en el servidor?

Aspecto	Guardar en Servidor	MinIO
Escalabilidad	Limitada por disco del servidor	Fácilmente escalable, puede usar múltiples discos/nodos
Rendimiento	Limitado por I/O del servidor	Optimizado para almacenamiento de objetos
Gestión	Archivos dispersos en carpetas	Organización por buckets, versionado, metadatos
Respaldo	Complicado, requiere scripts personalizados	Herramientas nativas de backup y replicación

Aspecto	Guardar en Servidor	MinIO
URLs públicas	Requiere configurar servidor web	URLs públicas automáticas con políticas S3
Compatibilidad	Específico del servidor	Compatible con API S3 (estándar de industria)
Migración a nube	Requiere reescritura	Fácil migración a AWS S3, Google Cloud Storage, etc.

Beneficios Específicos de MinIO en UTMentor

1. **Desacoplamiento**: Las fotos están separadas de la lógica del servidor API
2. **Persistencia**: Los archivos sobreviven a reinicios de contenedores
3. **Acceso directo**: Las URLs públicas permiten que el navegador cargue las imágenes directamente desde MinIO, sin pasar por el servidor API
4. **Preparación para producción**: Fácil migración a AWS S3 o servicios similares cambiando solo las credenciales

---

2. JWT: Sistema de Autenticación

¿Qué es JWT?

JWT (JSON Web Token) es un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información de forma segura entre dos partes como un objeto JSON.

En términos simples: Es como una "credencial digital" que el servidor genera y firma, y que el cliente presenta en cada petición para demostrar su identidad.

Estructura de un JWT

Un JWT está compuesto por tres partes separadas por puntos (`.`):

```
header.payload.signature
```

Ejemplo real:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTIzLCJyb2wiOiJhc2Vzb3I
iLCJpYXQiOjE2MzI0ODk2MDB9.kJIS5_gH9xVzFd8KqN2pL4mRn6Yh3Tc8Wj9
Xk2Qp1rY
```


1. Header (Encabezado)

```
{  "alg": "HS256",  "typ": "JWT"}
```

Especifica el algoritmo de cifrado (HS256 = HMAC SHA-256) y el tipo de token.

2. Payload (Carga útil)

```
{  "id": 123,  "rol": "asesor",  "iat": 1632489600,  "exp": 1632493200}
```

Contiene los datos ("claims") que queremos transmitir:
- `id`: ID del usuario
- `rol`: Rol del usuario (asesor/asesorado)
- `iat`: Timestamp de emisión (issued at)
- `exp`: Timestamp de expiración

3. Signature (Firma)

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret_key
)
```

La firma garantiza que el token no ha sido manipulado. Se genera usando el header, el payload y una clave secreta que solo el servidor conoce.

Implementación en el Código

1. Utilidades JWT (`api/utils/jwt.js`)

```
import jwt from "jsonwebtoken";const JWT_SECRET = process.env.JWT_SEC
RET;// Generar un token JWTexport function generarToken(payload) {
  return jwt.sign(payload, JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN || "1h",   });}
// Verificar y decodificar un token JWTexport function verificarToken(token) {
  try {
    return jwt.verify(token, JWT_SECRET);   } catch (err) {
    return null;  // Token inválido o expirado   }
}
```

Variables de entorno requeridas:

```
JWT_SECRET=tu_clave_secreta_super_segura_123456
JWT_EXPIRES_IN=1h
```

¿Por qué es importante JWT_SECRET?

La clave secreta es como la "contraseña maestra" que el servidor usa para:
1. **Firmar tokens**: Al crear un token, se firma con esta clave
2. **Verificar tokens**: Al recibir un token, se verifica que la firma coincida

IMPORTANTE: Si esta clave se compromete, un atacante podría generar tokens válidos falsificando cualquier identidad.

2. Proceso de Login (`api/controllers/authController.js`)

```
export async function login(req, res) {
  const { correo, password } = req.body;   try {
    // 1. Buscar usuario por correo      const [rows] = await mysqlPool.query
(
      "SELECT * FROM usuarios WHERE correo = ? LIMIT 1",          [correo]
    );      if (!rows || rows.length === 0) {
      return res.status(400).json({ error: "Correo o contraseña incorrectos"
});      }
    const usuario = rows[0];      // 2. Verificar contraseña con bcrypt      co
nst esValido = await bcrypt.compare(password, usuario.password_hash);
```

```
if (!esValido) {
        return res.status(400).json({ error: "Correo o contraseña incorrectos"
});      }
      // 3. Generar JWT       const token = generarToken({
        id: usuario.id_usuario,          rol: usuario.rol,       });       // 4. Enviar tok
en en header y respuesta       res.setHeader("Authorization", `Bearer ${token}
`);       return res.json({
        mensaje: "Login exitoso",          token,          usuario: {
          id: usuario.id_usuario,            nombre: usuario.nombre_completo,
rol: usuario.rol,          },       });    } catch (err) {
      console.error("Error en login:", err);       res.status(500).json({ error: "Err
or interno del servidor" });    }
}
```

Flujo de autenticación:

1. Usuario envía credenciales (correo + contraseña)
2. Servidor busca el usuario en la base de datos
3. Compara la contraseña usando bcrypt (nunca se almacenan contraseñas en texto plano)
4. Si es válida, genera un JWT con el ID y rol del usuario
5. Devuelve el token al cliente
6. Cliente guarda el token (usualmente en localStorage o sessionStorage)

3. Middleware de Autenticación (`api/middleware/authMiddleware.js`)

```
import { verificarToken } from "../utils/jwt.js";export function authMiddleware(r
eq, res, next) {
   try {
      // 1. Obtener el header de autorización       const authHeader = req.head
ers.authorization;       if (!authHeader) {
        return res.status(401).json({
          error: "Authorization header faltante",            detalle: "Debe enviar
'Authorization: Bearer <token>'"          });       }
```

```
    // 2. Validar formato: "Bearer token"    const partes = authHeader.split
(" ");    if (partes.length !== 2 || partes[0] !== "Bearer") {
      return res.status(400).json({
        error: "Formato de Authorization inválido",          detalle: "Use 'Be
arer <token>'"        });    }
    const token = partes[1];    // 3. Verificar token    const decoded = ver
ificarToken(token);    if (!decoded) {
      return res.status(403).json({
        error: "Token inválido o expirado"        });    }
    // 4. Guardar datos del usuario en req    req.usuario = decoded;    //
5. Continuar con la petición    next();  } catch (error) {
    console.error("Error en authMiddleware:", error);    return res.status(50
0).json({
      error: "Error interno en autenticación"    });  }
}
```

¿Cómo funciona el middleware?

Este middleware se ejecuta ANTES de que una ruta protegida procese la petición:

```
// Ruta protegidarouter.get("/perfil", authMiddleware, obtenerPerfil);
```

Secuencia:

1. Cliente envía petición con header: `Authorization: Bearer eyJhbGc...`
2. Middleware extrae el token
3. Verifica la firma y validez del token
4. Si es válido, decodifica el payload y lo guarda en `req.usuario`
5. La función `obtenerPerfil` puede acceder a `req.usuario.id` y `req.usuario.rol`
6. Si el token es inválido, retorna error 403 y detiene la petición

4. Middleware de Roles (`api/middleware/roleMiddleware.js`)

```
export function roleMiddleware(rolesPermitidos = []) {
  return (req, res, next) => {
    const { rol } = req.usuario; // viene del authMiddleware    if (!rolesPermi
tidos.includes(rol)) {
```

```
        return res.status(403).json({ error: "Acceso denegado" });        }
    next();    };}
```

Uso combinado de middlewares:

```
import { authMiddleware } from "../middleware/authMiddleware.js";import { rol
eMiddleware } from "../middleware/roleMiddleware.js";// Solo asesores puede
n actualizar su perfilrouter.put(
  "/asesores/:id",
  authMiddleware,                    // 1. Verificar que esté autenticado  roleMiddlew
are(['asesor']),        // 2. Verificar que sea asesor  actualizarAsesor
// 3. Ejecutar la función);
```

Configuración de CORS para JWT

En `api/server.js`, se configura CORS para permitir el header `Authorization`:

```
app.use(cors({
  origin: true,  methods: ['GET','POST','PUT','DELETE','OPTIONS'],  allowedHea
ders: ['Content-Type', 'Authorization', 'Accept'],  exposedHeaders: ['Authoriza
tion']
}));
```

¿Por qué es necesario esto?

Los navegadores por seguridad bloquean headers personalizados en peticiones CORS. Al especificar:
- `allowedHeaders`: Permite que el cliente envíe el header `Authorization`
- `exposedHeaders`: Permite que el cliente lea el header `Authorization` en la respuesta

Flujo Completo de Autenticación JWT

```
┌──────────────────┐                    ┌──────────────────┐
│   Cliente   │                    │   Servidor   │
│  (Browser)  │                    │     API     │
└──────────┬──────────┘                    └──────────┬──────────┘
      │                                  │
```

```
      │  1. POST /api/auth/login                        │
      │     { correo, password }                        │
      ├─────────────
```

```
──┤
  │                                   │
```

Seguridad del Sistema JWT

¿Qué NO almacenar en el payload del JWT?

❌ **Nunca incluir:**
- Contraseñas
- Números de tarjeta de crédito
- Información sensible o privada

✅ **Sí incluir:**
- ID del usuario
- Rol/permisos
- Información pública no sensible

Razón: El payload del JWT está codificado en Base64, no encriptado. Cualquiera que obtenga el token puede decodificar y leer el payload. La firma solo garantiza que no ha sido modificado, no que sea secreto.

Buenas Prácticas Implementadas

1. **Tiempo de expiración corto** (`1h`): Reduce ventana de ataque si un token es robado
2. **Clave secreta fuerte**: `JWT_SECRET` debe ser compleja y guardada en variables de entorno
3. **HTTPS en producción**: Previene que tokens sean interceptados
4. **Validación en cada petición**: El middleware verifica el token en cada ruta protegida
5. **No almacenar información sensible**: Solo ID y rol en el payload

---

3. Integración y Flujo de Trabajo

Caso de Uso Completo: Actualizar Foto de Perfil de Asesor

Este ejemplo muestra cómo JWT y MinIO trabajan juntos:

```
┌──────────────┐                          ┌──────────────┐
│   Cliente    │                          │   Servidor   │
└──────┬───────┘                          └──────┬───────┘
       │                                         │
       │  1. Login previo                        │
       │     POST /api/auth/login                │
       │     Obtiene: token = "eyJhbGc..."       │
       │                                         │
       │  2. Seleccionar foto en el navegador    │
       │     <input type="file">                 │
       │                                         │
       │  3. Subir foto con autenticación        │
       │     POST /api/usuarios/asesores/123/foto │
       │     Headers:                            │
       │       Authorization: Bearer eyJhbGc...  │
       │       Content-Type: multipart/form-data │
       │     Body: FormData con archivo          │
       ├─────────────
```

```
      │  7. Respuesta con URL                          │
      │    {                                  │
      │      message: "Foto actualizada",               │
      │      url: "http://localhost:9000/..."           │
      │    }                                  │
      │<──────────────
```

En `api/routes/usuarioRoutes.js`, podemos ver cómo se protegen diferentes endpoints:

```
// Endpoint público - no requiere autenticaciónrouter.get("/asesores", listarAs
esores);// Endpoint protegido - requiere autenticaciónrouter.post(
  "/asesores/:id/foto",
  authMiddleware,            // Solo usuarios autenticados  upload.single('foto'),
// Con archivo  subirFotoPerfil
);// Endpoint con roles - solo asesoresrouter.put(
  "/asesores/:id",
  authMiddleware,            // Usuario autenticado  roleMiddleware(['asesor']),
// Y debe ser asesor  actualizarAsesor
);
```

---

4. Beneficios y Aportes al Proyecto

Beneficios de MinIO

1. **Escalabilidad y Rendimiento**

- **Separación de responsabilidades**: El servidor API no tiene que servir archivos estáticos
- **Carga directa**: El navegador carga las imágenes directamente desde MinIO, no desde el API
- **Optimización**: MinIO está optimizado específicamente para almacenamiento de objetos

Sin MinIO:

```
Cliente → [Solicita imagen] → API → [Lee disco] → API → Cliente
        (El API es intermediario innecesario)
```

Con MinIO:

```
Cliente → [Solicita imagen] → MinIO → Cliente
```

```
        (Carga directa, más rápida)
```

2. **Gestión Profesional de Archivos**

- URLs públicas automáticas
- Políticas de acceso configurables
- Metadatos en cada objeto
- Versionado de archivos
- Políticas de lifecycle (eliminación automática de archivos antiguos)

3. **Preparación para Producción**

Cambiar de MinIO local a AWS S3 requiere solo actualizar credenciales:

```
// Desarrollo (MinIO)const minioClient = new Minio.Client({
  endPoint: 'localhost',  port: 9000,  accessKey: 'minioadmin',  secretKey: 'mini
oadmin',});// Producción (AWS S3)const minioClient = new Minio.Client({
  endPoint: 's3.amazonaws.com',  accessKey: process.env.AWS_ACCESS_KE
Y,  secretKey: process.env.AWS_SECRET_KEY,});
```

El código del controlador permanece **idéntico**.

4. **Consola de Administración**

MinIO incluye una interfaz web (puerto 9001) donde puedes:
- Ver todos los archivos subidos
- Descargar/eliminar archivos manualmente
- Configurar políticas de acceso
- Monitorear uso de almacenamiento
- Gestionar usuarios y permisos

Beneficios de JWT

1. **Stateless (Sin Estado)**

El servidor no necesita almacenar sesiones:

Sesiones tradicionales:

```
Cliente ──[ID de sesión: abc123]──> Servidor
                                  ↓
                            [Busca en BD/Redis]
                            ¿Sesión válida?
                            ¿Usuario activo?
```

JWT:

```
Cliente ──[JWT con toda la info]──> Servidor
                                  ↓
                            [Verifica firma]
                            ¡Listo! (sin BD)
```

Ventajas:
- No requiere almacenamiento de sesiones en servidor
- Escalable horizontalmente (múltiples servidores sin compartir sesiones)
- Menor carga en base de datos

2. **Descentralización**

El token contiene toda la información necesaria:

```
// Token decodificado{
  id: 123,  rol: "asesor",  iat: 1632489600,  exp: 1632493200}
```

El servidor puede tomar decisiones sin consultar la base de datos en cada petición.

3. **Compatibilidad con Microservicios**

Si en el futuro UTMentor crece y se divide en microservicios:

```
┌──────────────────┐
│  API Auth   │ ──[Genera JWT]──┐
└──────────────────┘                 │
```

```
                    ↓
┌──────────────────┐       ┌──────────────────────┐
│ API Usuarios │ ←──────────│  Cliente con   │
└──────────────────┘   ↑   │     JWT      │
            │   └──────────────────────┘
                              │
┌──────────────────┐   │         ↓
│ API Asesorías │ ←──────┴──────[Mismo JWT]
└──────────────────┘

Cada servicio puede verificar el JWT independientemente
```

4. **Móviles y SPAs**

JWT es ideal para aplicaciones de página única (React, Vue, Angular) y apps móviles:

- No depende de cookies
- Funciona con cualquier cliente HTTP
- Fácil de implementar en móviles (React Native, Flutter, etc.)

5. **Seguridad Mejorada**

- **No vulnerable a CSRF**: No usa cookies de sesión
- **Expiración automática**: Tokens expiran después de 1 hora
- **Firma criptográfica**: Imposible falsificar sin la clave secreta
- **Información autocontenida**: Reduce consultas a BD

Aportes Conjuntos al Proyecto UTMentor

1. **Arquitectura Moderna y Profesional**

- **Separación de concerns**: Autenticación (JWT), almacenamiento (MinIO), lógica (API)
- **Estándares de industria**: JWT (RFC 7519), S3-compatible storage
- **Preparado para escalar**: Tanto MinIO como JWT escalan horizontalmente

2. **Experiencia de Usuario Mejorada**

- **Carga rápida de imágenes**: MinIO sirve archivos optimizadamente
- **Autenticación fluida**: JWT permite mantener sesión sin cookies
- **Sin recargas**: El frontend puede mantener el token y hacer peticiones sin interrupciones

3. **Seguridad Robusta**

```
// Ejemplo de endpoint ultra-segurorouter.put(
  "/asesores/:id/foto",
  authMiddleware,                  // ✓ Usuario autenticado  roleMiddleware(['ases
or']),        // ✓ Rol correcto  validarOwnership,                // ✓ Es dueño del rec
urso  upload.single('foto'),            // ✓ Archivo válido  validarTamanio,
// ✓ Tamaño permitido  subirFotoPerfil
);
```

4. **Mantenibilidad**

- **Código modular**: JWT en `utils/`, MinIO en `config/`
- **Middleware reutilizable**: `authMiddleware` se usa en múltiples rutas
- **Fácil testing**: Funciones puras y bien separadas

5. **Reducción de Costos**

- **Menos carga en servidor**: MinIO maneja archivos, el API solo lógica
- **Menos consultas a BD**: JWT reduce necesidad de verificar sesiones
- **Caching eficiente**: Las imágenes en MinIO pueden cachearse fácilmente

Comparación: Antes vs Después

Aspecto	Sin MinIO/JWT	Con MinIO/JWT
Almacenamiento de fotos	Carpeta local en servidor	Bucket S3-compatible escalable
Autenticación	Sesiones en memoria/BD	JWT stateless

Aspecto	Sin MinIO/JWT	Con MinIO/JWT
Seguridad	Cookies, vulnerable a CSRF	Tokens, más seguro
Escalabilidad	Limitada, sesiones compartidas	Horizontal, sin estado
Carga de imágenes	A través del API	Directa desde MinIO
Preparación cloud	Requiere refactorización	Listo para AWS/GCP
Apps móviles	Complicado con cookies	Nativo con JWT

Casos de Uso en UTMentor

Caso 1: Asesor Sube Foto de Perfil

```
1. Asesor hace login → Obtiene JWT
2. Asesor selecciona foto en su perfil
3. Frontend envía foto + JWT al API
4. API verifica JWT (es asesor)
5. API sube foto a MinIO
6. MinIO devuelve URL pública
7. API actualiza BD con URL
8. Frontend muestra nueva foto desde MinIO
```

Beneficio: Foto cargada rápidamente, sin sobrecargar el API.

Caso 2: Asesorado Busca Asesores

```
1. Asesorado hace login → Obtiene JWT
2. Asesorado busca asesores
3. Frontend solicita lista con JWT
4. API verifica JWT (autenticado)
5. API devuelve asesores con URLs de MinIO
6. Frontend carga fotos directamente desde MinIO
```

Beneficio: Lista de asesores se carga rápido, fotos en paralelo desde MinIO.

Caso 3: Protección de Recursos

```
1. Usuario intenta actualizar perfil de otro usuario
2. Frontend envía petición con JWT
3. authMiddleware verifica JWT
4. Controlador compara req.usuario.id con :id del endpoint
5. Si no coincide, retorna 403 Forbidden
```

Beneficio: Nadie puede modificar datos de otros usuarios.

Conclusión

La implementación de **MinIO** y **JWT** en UTMentor aporta:

MinIO:

✅ Almacenamiento escalable y profesional de archivos

✅ Separación de responsabilidades (archivos vs lógica)

✅ URLs públicas y acceso optimizado

✅ Compatible con estándares de industria (S3)

✅ Fácil migración a servicios cloud

JWT:

✅ Autenticación stateless y escalable

✅ Seguridad robusta con firma criptográfica

✅ Ideal para SPAs y aplicaciones móviles

✅ Reducción de carga en base de datos

✅ Compatible con arquitecturas de microservicios

Juntos:

🚀 **Arquitectura moderna y profesional**

🚀 **Preparado para escalar y crecer**

🚀 **Seguridad de nivel empresarial**

🚀 **Experiencia de usuario optimizada**

🚀 Código mantenible y testeable

Este diseño posiciona a UTMentor como una aplicación de nivel profesional, lista para evolucionar y escalar según las necesidades del negocio.